

Progetto 1 — KernelExplorer

Relazione tecnica
Un modulo kernel Linux interattivo
per l'esplorazione delle strutture del sistema operativo

Corso:	Sistemi Operativi e Sicurezza Informatica
Docente:	Prof. Francesco Fontanella
Studente:	Andrea Molle 78101
Anno accademico:	2025/2026
Modulo:	kexplorer

Indice

Parte I — Fondamenti teorici

1. I moduli kernel Linux
2. Il filesystem /proc
3. Spazio utente e spazio kernel
4. La task_struct e la rappresentazione dei processi
5. L'albero dei processi in Linux
6. Le linked list del kernel Linux
7. La gestione della memoria nel kernel
8. Il kernel log e printk

Parte II — Descrizione dell'implementazione

9. Architettura generale
10. Scelte implementative per fase
11. Gestione degli errori e dei casi limite
12. Testing

Parte III — Diario dell'interazione con l'IA

13. Strumenti e modalità di utilizzo
14. Schede di interazione per fase
15. Analisi critica e bilancio finale

Conclusioni

Riferimenti

Parte I — Fondamenti teorici

1. I moduli kernel Linux

Cos'è

Un *Loadable Kernel Module* (LKM) è un frammento di codice oggetto che può essere collegato dinamicamente al kernel Linux mentre questo è in esecuzione, estendendone le funzionalità senza richiedere ricompilazione né riavvio. Una volta caricato, il modulo opera all'interno dello spazio di indirizzamento del kernel: condivide con esso lo stack, le strutture dati e le API, e gode degli stessi privilegi.

Il kernel Linux è *monolitico*: tutto il codice di sistema gira in kernel mode e in un unico spazio di indirizzamento. La modularità dei LKM è una scelta architetturale che mitiga il principale svantaggio del modello monolitico — la dimensione e rigidità del binario — senza abbandonarne i vantaggi di efficienza (assenza di context switch per le chiamate interne, mancanza di IPC tra sotto-sistemi).

Come funziona

Ogni modulo definisce due funzioni speciali: una di *inizializzazione* (decorata con `__init`, registrata via `module_init()`) e una di *uscita* (`__exit`, `module_exit()`). La prima viene invocata automaticamente da `insmod/modprobe` al caricamento del modulo nel kernel; la seconda da `rmmmod` al momento della rimozione. In mezzo, il modulo è parte integrante del kernel e può essere chiamato in risposta a system call, interrupt, eventi di I/O, operazioni su file di pseudo-filesystem, ecc.

Il caricamento avviene attraverso la system call `finit_module(2)`: il loader del kernel verifica la firma (se obbligatoria), risolve i simboli importati rispetto a quelli esportati dal kernel via `EXPORT_SYMBOL` o `EXPORT_SYMBOL_GPL`, applica le rilocalizzazioni e infine invoca la funzione registrata da `module_init`. La rimozione segue il percorso inverso: si controlla che il *reference count* del modulo sia zero, si invoca la funzione di exit, si liberano i simboli.

Perché nel mio progetto

Il progetto è realizzato come modulo e non come patch al kernel per tre ragioni concrete: **(i)** il ciclo di sviluppo è incomparabilmente più rapido (modifica → `make` → `insmod` richiede pochi secondi, contro decine di minuti per ricompilare il kernel); **(ii)** in caso di errore grave durante lo sviluppo, la rimozione del modulo (o nel peggiore dei casi un reboot della VM) ripristina lo stato iniziale, mentre una patch al kernel difettosa lo lascerebbe permanentemente compromesso; **(iii)** la consegna del progetto è un singolo file `kexplorer.ko` che si carica su qualsiasi sistema Linux compatibile, senza richiedere modifiche intrusive all'host di chi corregge.

Nel codice, l'infrastruttura del modulo è concentrata nelle due funzioni `kexplorer_init` (che salva il valore iniziale di `jiffies`, alloca il buffer di output con `kmalloc` e crea l'entry `/proc/kexplorer`) e `kexplorer_exit` (che rimuove l'entry `/proc` e libera il buffer). Le macro `MODULE_LICENSE("GPL")`, `MODULE_AUTHOR` e `MODULE_DESCRIPTION` popolano i metadati consultabili con `modinfo`: la dichiarazione GPL non è formale, è obbligatoria per poter accedere a

simboli del kernel esportati con `EXPORT_SYMBOL_GPL` (come `init_task`).

2. Il filesystem `/proc`

Cos'è

`/proc` è uno *pseudo-filesystem* interno al kernel: i file che vi appaiono non risiedono su disco, ma vengono generati dinamicamente dal kernel ogni volta che si effettua una `read` o una `write`. Nato in Linux come porting del `/proc` di System V e poi esteso ben oltre il suo scopo originale, ospita oggi sia informazioni sui processi (`/proc/<pid>/...`) sia parametri tunabili del kernel (`/proc/sys/...`) sia viste su stato hardware e statistiche (`/proc/cpuinfo`, `/proc/meminfo`, ecc.).

L'idea fondante è la filosofia Unix «*everything is a file*»: le informazioni del kernel sono rese accessibili con le stesse operazioni con cui si accede ai file regolari (`open`, `read`, `write`, `close`), in modo che qualunque programma — dalla shell ai linguaggi di scripting — possa interagire con il kernel senza bisogno di API specifiche.

Come funziona

Un modulo crea un'entry in `/proc` chiamando `proc_create(name, mode, parent, &proc_ops)`. La struttura `proc_ops` (in passato `file_operations`) contiene puntatori a funzione: i campi `.proc_read` e `.proc_write` sono i *callback* che il kernel invoca quando un processo dello spazio utente legge o scrive sull'entry. Niente è memorizzato «sul file»: la `read` esegue codice del kernel che produce byte da restituire all'utente, e la `write` esegue codice del kernel che consuma i byte ricevuti.

Esistono alternative a `/proc` per l'interazione kernel/utente: `sysfs` (pensato per esporre la gerarchia dei device come oggetti tipizzati), i *device file* con `ioctl` (numerose operazioni binarie), *netlink* (canali di messaggi asincroni), `debugfs` (per debugging e tracing). `/proc` resta la scelta naturale per interfacce testuali ad uso umano e per piccoli moduli didattici.

Perché nel mio progetto

`/proc` è la scelta ideale per KernelExplorer perché serve un canale *bidirezionale* (comandi in scrittura, risultati in lettura), *testuale* (così si interagisce con la sola coppia `echo/cat`) e *semplice* (un solo file, nessun protocollo). Le alternative sarebbero state più complesse: `sysfs` impone un attributo per «valore» e non si presta bene a un dispatcher di comandi; un device file con `ioctl` avrebbe richiesto la definizione di una ABI binaria con codici operazione; *netlink* sarebbe stato sovradimensionato.

Nel codice, `proc_write()` alloca un buffer kernel con `kmalloc`, copia il comando dallo spazio utente con `copy_from_user`, passa la stringa al dispatcher e libera la memoria con `kfree`. `proc_read()` fa l'operazione speculare: copia il buffer di output verso lo spazio utente con `copy_to_user` e usa un flag `completed` per segnalare EOF nella seconda chiamata (altrimenti `cat` chiamerebbe `read` all'infinito).

3. Spazio utente e spazio kernel

Cos'è

Su architetture moderne il processore distingue almeno due livelli di privilegio: sull'x86-64 sono i quattro *ring*, dei quali Linux usa il ring 0 per il kernel e il ring 3 per i processi utente. Ad ogni livello corrisponde uno *spazio di indirizzamento virtuale*: la metà alta dei 64 bit (a partire da `0xffff800000000000` sull'x86-64) è riservata al kernel e mappata uguale per tutti i processi; la metà bassa è propria di ciascun processo utente.

Questa separazione esiste per due motivi: **protezione** — un errore o un comportamento malevolo in un processo non può corrompere il kernel né altri processi — e **controllo dell'accesso alle risorse**: solo il kernel può eseguire istruzioni privilegiate (I/O, modifica delle page table, gestione degli interrupt).

Come funziona

Un processo entra nel kernel solo attraverso punti di accesso controllati: *system call* (istruzione `syscall` su x86-64), trap di eccezione, interrupt hardware. Al passaggio, la CPU salva lo stato, cambia modalità da ring 3 a ring 0 e salta a un *entry point* predefinito; al ritorno fa l'operazione inversa.

Il kernel non può *dereferenziare direttamente* un puntatore che proviene dallo spazio utente. Anche se l'indirizzo virtuale fosse leggibile in teoria, farlo significherebbe esporre il kernel a un page fault non gestibile (la pagina potrebbe non essere presente in memoria) e bypassare i controlli di permesso. Per questo esistono funzioni dedicate: `copy_from_user(dst, src, n)` verifica che `src` sia un indirizzo valido nello spazio del processo chiamante, gestisce eventuali fault in modo sicuro e copia `n` byte verso il buffer kernel `dst`. `copy_to_user` è l'opposto. Entrambe restituiscono il numero di byte *non* copiati: 0 indica successo.

Sui kernel recenti è attiva anche *SMAP* (Supervisor Mode Access Prevention), che genera un page fault se il kernel tenta di accedere a un indirizzo utente senza prima abilitare l'accesso. Le funzioni `copy_*_user` incapsulano la disabilitazione e riabilitazione di SMAP con macro come `user_access_begin/end`.

Perché nel mio progetto

La comunicazione attraverso `/proc/kexplorer` attraversa due volte il confine fra spazio utente e spazio kernel: una in scrittura, una in lettura. Nel codice:

```
/* proc_write: spazio utente -> kernel */
k_mem = kmalloc(count + 1, GFP_KERNEL);
if (copy_from_user(k_mem, usr_buf, count)) {
    kfree(k_mem);
    return -EFAULT;
}

/* proc_read: kernel -> spazio utente */
if (copy_to_user(usr_buf, output_buffer, output_len))
    return -EFAULT;
```

Un tentativo di accedere direttamente con `memcpy` al buffer utente provocherebbe, nei casi più benigni, un *kernel oops* con stack trace in `dmesg`; nei casi peggiori una corruzione silenziosa di pagine utente. Il valore di ritorno di `-EFAULT` è anch'esso obbligatorio: il kernel lo traduce in errore della `syscall` che ha innescato la `read/write`.

4. La task_struct e la rappresentazione dei processi

Cos'è

Il *Process Control Block* (PCB) è la struttura dati con cui un sistema operativo rappresenta un processo. In Linux il PCB è implementato da `struct task_struct`, definita in `<linux/sched.h>`. È probabilmente la struttura dati più grande del kernel: contiene centinaia di campi, perché aggrega tutto ciò che il kernel deve sapere su un'entità eseguibile.

I campi più rilevanti dal punto di vista didattico sono: `pid` (identificatore numerico del task), `comm` (nome dell'eseguibile, 16 caratteri inclusi terminatore), `__state` ex `state` (stato corrente: `TASK_RUNNING`, `TASK_INTERRUPTIBLE`, ecc.), `parent` e `real_parent` (legami di parentela), `children` e `sibling` (liste dei figli e dei fratelli), `mm` (descrittore della memoria virtuale), `files` (tabella dei file aperti), `cred` (credenziali e capabilities).

Come funziona

Linux non distingue, a livello di kernel, fra *processi* e *thread*: tutti i task sono rappresentati dalla stessa `task_struct`. Quel che cambia è come la struttura viene istanziata. Un `fork()` crea una nuova `task_struct` con uno spazio di indirizzamento separato (`mm` diverso); una `pthread_create()` usa internamente la system call `clone()` con flag `CLONE_VM | CLONE_FILES | CLONE_SIGHAND`, che condivide spazio di indirizzamento e descrittori. Il `tgid` (*thread group id*) raggruppa i thread di uno stesso processo POSIX.

I task del sistema sono organizzati in una *doubly-linked list* globale, attraversabile con la macro `for_each_process(task)`, e in un *albero di parentela* formato dai campi `children` (testa della lista dei figli) e `sibling` (link al fratello successivo).

Perché nel mio progetto

Il comando `task PID` recupera la `task_struct` a partire dal PID e ne stampa tre campi (`comm`, `pid`, `stato`). Il percorso `PID→task` passa attraverso due funzioni: `find_vpid()` ottiene la struct `pid` nel *PID namespace* corrente, e `pid_task()` estrae la `task_struct` associata. La doppia indirazione esiste perché in presenza di PID namespace lo stesso PID numerico può identificare task diversi.

Un dettaglio implementativo importante è la differenza fra `task->state` (kernel < 5.14, tipo `volatile long`) e `task->__state` (kernel ≥ 5.14, tipo `unsigned int`), introdotta dal commit che ha separato lo stato della scheduling class. Per garantire portabilità fra Ubuntu 20.04 e Ubuntu 22.04 ho definito una macro `KE_TASK_STATE(t)` che seleziona il campo corretto in base a `LINUX_VERSION_CODE`.

5. L'albero dei processi in Linux

Cos'è

Ogni processo Linux (eccetto la radice) discende da un genitore attraverso `fork()` o `clone()`. Questo rapporto induce una struttura ad albero, di cui la radice è `init_task` — il task associato al *processo swapper* (PID 0). Tutti gli altri task sono discendenti diretti o indiretti di `init_task`.

Come funziona

L'albero è implementato con due liste agganciate in ogni `task_struct`:

- `children` è la *testa* della lista dei figli del task. È una `struct list_head`.
- `sibling` è il *link* che ciascun figlio usa per agganciarsi alla lista `children` del padre, oltre

che per scorrere i fratelli.

Per attraversare l'albero esistono due strategie concettualmente diverse: **iterazione lineare** con `for_each_process(task)`, che scorre tutti i task del sistema indipendentemente dalla parentela (utile per elenchi piatti tipo `ps -el`); e **attraversamento gerarchico in DFS**, che a partire dalla radice visita un nodo, scende sui suoi figli, poi sui figli dei figli, eccetera.

Perché nel mio progetto

Implemento entrambe le strategie nei comandi `list` e `tree` per mostrare la differenza in modo evidente. Il DFS è realizzato con una funzione ricorsiva `dfs_iterate()` che stampa il task corrente e poi si invoca su ciascun figlio attraverso una `list_for_each` sulla lista `children`:

```
list_for_each(list, &task->children) {
    child = list_entry(list, struct task_struct, sibling);
    dfs_iterate(child);
}
```

`list_entry` applica il classico trucco `container_of`: dato un puntatore al campo `sibling`, sottrae l'offset di quel campo nella `task_struct` per ottenere il puntatore alla struttura contenitrice.

L'output di `tree` contiene in genere *più* righe di quello di `list` (e anche di `ps -el`) perché include i *thread del kernel* che, sebbene non siano processi nel senso POSIX, hanno una `task_struct` e fanno parte dell'albero. Anche `list` li mostra: il confronto fa emergere la differenza fra la nozione utente di «processo» e quella kernel di «task».

Nota sulla profondità della ricorsione: lo stack del kernel è tipicamente di 8 KB (16 KB sulle architetture a 64 bit moderne). L'albero dei processi reali ha profondità dell'ordine delle decine di livelli, quindi la ricorsione è sicura. In contesti più estremi sarebbe opportuno trasformarla in iterativa con uno stack esplicito.

6. Le linked list del kernel Linux

Cos'è

Le *linked list intrusive* del kernel sono un pattern di implementazione delle liste doppiamente concatenate che capovolge il modello tradizionale. In una lista tradizionale ogni nodo è un oggetto fatto di dato + puntatori; nelle liste intrusive del kernel il nodo è il dato, e i puntatori sono un piccolo «link» incorporato come campo della struttura utente.

Come funziona

Il link è la struttura `struct list_head { struct list_head *next, *prev; }`, definita in `<linux/list.h>`. Si dichiara una lista vuota con la macro `LIST_HEAD(name)`, che produce una `struct list_head` circolare con `next` e `prev` che puntano a sé stessa. Inserimento e rimozione sono operazioni $O(1)$ grazie alla natura circolare e doppia.

Le macro di iterazione sfruttano `container_of` per risalire dalla `list_head` al contenitore: `list_for_each_entry(pos, head, member)` dichiara `pos` del tipo del contenitore e a ogni iterazione lo aggiorna. `list_for_each_entry_safe(pos, tmp, head, member)` aggiunge un cursore `tmp` che custodisce il nodo successivo, consentendo di rimuovere `pos` durante l'iterazione senza invalidare il loop.

Il vantaggio progettuale: lo stesso pattern serve tutte le liste del kernel (task list, runqueue dello scheduler, lista dei filesystem registrati, ecc.) senza alcuna API specifica per tipo di dato. Una sola implementazione, fortemente collaudata e ottimizzata, in tutto il kernel.

Perché nel mio progetto

Il comando `collatz` definisce una `struct collatz_node` con un campo `value` di tipo `unsigned long` e un campo `list` di tipo `struct list_head`:

```
struct collatz_node {
    unsigned long value;
    struct list_head list;
};

LIST_HEAD(collatz_list);
/* per ogni elemento della sequenza: */
node = kmalloc(sizeof(struct collatz_node), GFP_KERNEL);
node->value = n;
INIT_LIST_HEAD(&node->list);
list_add_tail(&node->list, &collatz_list);

/* iterazione */
list_for_each_entry(ptr, &collatz_list, list)
    /* uso ptr->value */;

/* deallocazione sicura durante iterazione */
list_for_each_entry_safe(ptr, next, &collatz_list, list) {
    list_del(&ptr->list);
    kfree(ptr);
}
```

Avrei potuto implementare una linked list classica con un `struct node { unsigned long v; struct node *next; }`, ma sarebbe stato un anti-pattern in codice kernel: l'idea del progetto è proprio toccare con mano le strutture dati del kernel.

7. La gestione della memoria nel kernel

Cos'è

Nel kernel l'allocazione dinamica di piccole quantità di memoria avviene tipicamente con la coppia `kmalloc()/kfree()`, l'analogo kernel di `malloc()/free()`. Le differenze sono però sostanziali: `kmalloc` alloca memoria *fisicamente contigua*, ha vincoli di dimensione (di solito limitata a 4 MB), e richiede di specificare un *flag di allocazione* di tipo `gfp_t` (*Get Free Pages*) che descrive il contesto in cui l'allocatore può lavorare.

Come funziona

`kmalloc` è costruito sopra il *SLUB allocator*, che gestisce *slab* di oggetti preallocati di taglie comuni. Una `kmalloc(sizeof(struct foo), GFP_KERNEL)` trova lo slab della taglia più piccola sufficiente e ne ritorna un oggetto.

Il flag `GFP_KERNEL` è il più comune e indica che il chiamante è in *contesto processo* e può essere messo in sleep mentre l'allocatore aspetta che si liberi memoria. In contesti dove non si può dormire (interrupt handler, tasklet, atomic context) si usa `GFP_ATOMIC`, che non blocca ma può fallire più facilmente.

Un *memory leak* nello spazio utente è circoscritto al processo: il sistema ne riprende il controllo alla terminazione. Nel kernel invece la memoria leakata resta sottratta al sistema fino al reboot. Pertanto la regola fondamentale del codice kernel è: *ogni allocazione deve avere un cammino di deallocazione, su ogni ramo di esecuzione, inclusi quelli di errore*.

Perché nel mio progetto

Il modulo effettua allocazioni in tre punti, e per ciascuno c'è una controparte di deallocazione esplicita:

- **Buffer di output:** allocato in `kexplorer_init()`, liberato in `kexplorer_exit()`.
- **Buffer temporaneo del comando:** allocato in `proc_write()`, liberato nello stesso scope prima del ritorno (anche sui rami di errore, per esempio se `copy_from_user` fallisce).
- **Nodi della linked list di Collatz:** allocati uno per elemento della sequenza, liberati tutti subito dopo la stampa con `list_for_each_entry_safe`. La macro `safe` è obbligatoria per il pattern «rimuovi durante l'iterazione».

Per *verificare* l'assenza di leak ho due strategie: **(a)** il modulo stampa nel kernel log il numero di nodi deallocati ad ogni `collatz`, in modo da confrontarlo con il numero di elementi della sequenza; **(b)** dopo aver caricato/rimosso il modulo molte volte e richiamato i comandi, `dmesg` non deve mostrare warning di tipo «leaked» e `/proc/slabinfo` non deve mostrare slab anomale (richiede `CONFIG_SLUB_DEBUG`). Su kernel con KASAN o kmemleak abilitati la verifica è automatica.

8. Il kernel log e printk

Cos'è

`printk()` è la funzione di logging del kernel: produce stringhe formattate (come `printf`) ma le invia al *kernel log ring buffer*, una struttura circolare in memoria del kernel di dimensione configurabile (tipicamente 1–16 MB). Il comando `dmesg` stampa il contenuto corrente del buffer; `journald` e `syslog` lo consumano e lo persistono su file.

Come funziona

`printk` non può usare `printf` perché quest'ultimo è una funzione della libc, che non esiste nel kernel: nel kernel non c'è uno *standard output*, non c'è il filesystem montato in tutte le fasi, e l'I/O verso terminale non è disponibile. `printk` scrive in memoria — non blocca su disco — quindi è chiamabile anche da interrupt handler.

Le stringhe possono essere prefissate da un livello di priorità: `KERN_EMERG`, `KERN_ALERT`, `KERN_CRIT`, `KERN_ERR`, `KERN_WARNING`, `KERN_NOTICE`, `KERN_INFO`, `KERN_DEBUG`. Il livello determina se la stringa appare in console (oltre che in `dmesg`) e influenza il filtraggio da parte di `journald`. Sono macro che si *concatenano* alla stringa di formato — non sono argomenti.

Perché nel mio progetto

Uso `printk` in tre contesti: **(i)** messaggi informativi al caricamento e alla rimozione (`KERN_INFO`); **(ii)** conferma della deallocazione dei nodi di Collatz (`KERN_INFO`, utile per debug e per verifica dell'assenza di leak); **(iii)** errori non recuperabili lato modulo (allocazione fallita, `copy_from_user` fallita), marcati con `KERN_ERR`. Gli errori rilevati e gestiti che però sono *attesi* (input non valido dell'utente) non vanno nel log: tornano nel buffer di output, dove li vede l'utente.

Parte II — Descrizione dell'implementazione

9. Architettura generale

Il modulo è un singolo file `kexplorer.c` organizzato in sezioni nette delimitate da commenti banner. L'idea guida è la separazione fra *infrastruttura* (entry `/proc`, buffer di output, dispatcher) e *handler dei comandi* (una funzione per comando, indipendenti fra loro). Lo schema del flusso di esecuzione è il seguente:

```
utente                                /proc/kexplorer                                kernel
-----                                -----                                -----
echo cmd >  -- write() --> proc_write(): kmalloc + copy_from_user
                                     |
                                     v
                                dispatch_command(): parsing
                                     |
                                     v
                                cmd_<X>(): scrivo in output_buffer
                                     |
                                     v
                                kfree(buffer temporaneo)

cat          -- read() --> proc_read(): copy_to_user(output_buffer)
<output>     <-- bytes ---          flag completed = 1 (EOF)
```

Le funzioni handler scrivono nel singolo `output_buffer` globale (allocato in `kexplorer_init`), che `proc_read` ritorna integralmente alla successiva `cat`. Questo design è semplice ma ha un limite: non gestisce accessi concorrenti da più processi. Per un progetto didattico mono-utente la cosa è accettabile; in un modulo «da produzione» bisognerebbe proteggere `output_buffer` con uno spinlock o un mutex.

10. Scelte implementative per fase

Fase 1 — info, gcd, parametro buffer_size

Il comando `info` è la «hello world» del progetto: serve a verificare che l'infrastruttura `/proc` funzioni. Stampa quattro valori (`GOLDEN_RATIO_PRIME`, `jiffies`, `HZ`, `uptime` in secondi). L'`uptime` è calcolato come differenza fra il `jiffies` corrente e quello salvato in `kexplorer_init`, divisa per `HZ` (il numero di tick al secondo).

Per `gcd` uso `kstrtoul` (non `simple_strtoul`, deprecata) per la conversione delle stringhe, verificando il valore di ritorno per intercettare stringhe non numeriche, overflow e altre condizioni di errore. La funzione kernel `gcd()` definita in `<linux/gcd.h>` implementa l'algoritmo di Stein (GCD binario), efficiente perché lavora con shift e sottrazioni invece che divisioni.

Il parametro `buffer_size` è dichiarato con `module_param` e `permesse = 0` (non leggibile in `/sys/module/...`). La dimensione di default 4096 byte è una scelta che rispecchia la page size: si alloca esattamente una pagina, evitando frammentazione.

Fase 2 — task

Il punto delicato è la conversione PID→task in modo robusto. Uso `kstrtol` (con la «l» finale, non «ul») per intercettare esplicitamente PID negativi, e separo i tre possibili insuccessi:

- **stringa non numerica** → `kstrtol` ritorna non-zero;
- **PID negativo** → controllo esplicito `pid_value < 0`;
- **PID non esistente** → `find_vpid()` o `pid_task()` ritornano NULL.

Ognuno produce un messaggio di errore distinto, in modo che l'utente capisca cosa è andato storto. Un punto su cui ho riflettuto è l'*RCU lock*: in teoria, `find_vpid` dovrebbe essere chiamato dentro `rcu_read_lock()/rcu_read_unlock()`, perché la `task_struct` potrebbe essere liberata da un altro thread mentre la consultiamo. Per un modulo didattico ad uso interattivo (un comando alla volta da shell) il rischio è trascurabile, ma è bene esserne consapevoli e averlo come miglioramento futuro.

Fase 3 — list e tree, gestione del troncamento

Il punto critico della Fase 3 è il *troncamento del buffer*. Con un sistema che ha centinaia di task e un buffer di 4096 byte, una stampa non gestita produrrebbe un buffer overflow nel migliore dei casi, un buffer overrun nel peggiore. La strategia che ho adottato è uguale per `list` e `tree`:

- A ogni iterazione calcolo lo `space` rimasto come `buffer_size - output_len - reserve`, dove `reserve` è la lunghezza del messaggio di troncamento (`TRUNC_MSG`). In questo modo conservo sempre spazio sufficiente per il messaggio finale.
- `snprintf` ritorna la lunghezza che *sarebbe* stata scritta in assenza di limiti. Se `written >= space`, c'è stato troncamento: ripristino il `'\0'` al punto di scrittura per scartare il dato parziale e setto un flag `truncated`.
- Alla fine, se il flag è settato, accodo `TRUNC_MSG` una sola volta.

Per `tree` il flag è una variabile `static int dfs_truncated` a livello di file: serve perché la ricorsione deve poter interrompersi «dal di fuori». Ogni livello, prima di lavorare, controlla il flag e ritorna se è già settato; e nel loop sui figli interrompe la ricorsione appena rileva il troncamento. Risultato: il messaggio viene aggiunto una sola volta, da `cmd_tree`.

Fase 4 — collatz e linked list

L'algoritmo della sequenza di Collatz è semplice in sé. La parte interessante è l'*integrazione* con le linked list del kernel:

1. **Generazione:** ciclo `while(1)` che alloca un nodo, lo inizializza con `INIT_LIST_HEAD`, lo appende con `list_add_tail` e calcola il valore successivo. Esce dopo aver inserito il valore.
1. Se `kmalloc` fallisce a metà strada, rilascia tutti i nodi già inseriti prima di ritornare: niente leak neppure sul ramo d'errore.
2. **Stampa:** itera con `list_for_each_entry` mettendo `" -> "` fra gli elementi. Anche qui gestisco il troncamento: se durante la stampa il buffer si riempie, annoto il troncamento e interrompo l'iterazione di stampa (la *deallocazione* avviene comunque dopo, indipendentemente dal troncamento).
3. **Deallocazione:** `list_for_each_entry_safe + list_del + kfree` in un unico loop. Conto i nodi liberati e ne registro il numero nel kernel log con `printk(KERN_INFO)`: dopo aver eseguito `collatz 6, dmesg | tail` deve mostrare `"Collatz(6) - 9 nodes freed"` (la sequenza 6,3,10,5,16,8,4,2,1 ha 9 elementi).

11. Gestione degli errori e dei casi limite

La filosofia è: *l'utente non deve poter crashare il kernel*. Le tabelle sotto mostrano gli scenari d'errore considerati e la risposta del modulo.

Scenario	Reazione del modulo
Comando vuoto o sconosciuto	Messaggio «unknown command» nel buffer
gcd con meno di 2 argomenti	«usage: gcd <n1> <n2>»
gcd con argomenti non numerici	«invalid number 'xxx'»
task con argomento mancante	«usage: task <pid>»
task con PID negativo	«PID must be non-negative»
task con PID inesistente	«no task found for PID N»
task con argomento non numerico	«'xxx' is not a valid integer»
collatz con argomento ≤ 0 o non numerico	Messaggio di errore specifico nel buffer
list/tree con buffer troppo piccolo	Output troncato + messaggio «[output truncated, buffer full]»
kmallocc fallito in collatz	Rilascio dei nodi già inseriti, KERN_ERR in dmesg
copy_from_user fallita in proc_write	kfree del buffer temporaneo, ritorno -EFAULT

Per gli errori «previsti» (input dell'utente) la reazione è un messaggio nel buffer di output: l'utente è chi ha sbagliato, è giusto che lo veda lui. Per gli errori «di sistema» (allocazione fallita, ecc.) la reazione è un `printk(KERN_ERR)` più, dove possibile, il ripristino delle risorse parziali.

12. Testing

Per ogni fase la specifica fornisce un piano di test puntuale. Lo schema con cui ho proceduto è sempre il seguente:

1. `sudo dmesg -c` per pulire il kernel log prima della sessione, in modo che eventuali messaggi del modulo emergano chiaramente.
2. `make` per compilare; controllo l'assenza di warning oltre che di errori (compilare il modulo con `-Wall -Wextra` aiuta a scoprire promotion implicite e variabili non usate).
3. `sudo insmod kexplorer.ko; lsmod | grep kexplorer; dmesg | tail; ls /proc/kexplorer`.
4. Per ogni comando, sequenza `echo "cmd..." > /proc/kexplorer && cat /proc/kexplorer`, confrontando l'output con l'atteso. Per `list` e `tree`, confronto numerico con `ps -el | wc -l` e `ps -eLf | wc -l`.
5. Test di robustezza: argomenti non numerici, PID inesistente (`task 99999`), valori limite (`collatz 0`, `collatz -10`).
6. Test di stress: `collatz 113383` (sequenza di 248 elementi), esecuzioni ripetute, alternanza dei comandi.
7. Test del parametro: ricarico con `buffer_size=512` e verifico che `list` e `tree` producano output troncato col messaggio appropriato.
8. `sudo rmmod kexplorer; dmesg | tail` per accertarsi che non ci siano warning di memoria non liberata.

I test della specifica sono organizzati per fase in categorie (A=funzionale, B=robustezza, C=stress, D=parametro). Ho riempito tutte le colonne «Esito» di tutte le tabelle, in particolare ho verificato che: `collatz 6` produca 9 elementi, `collatz 15` ne produca 18, `collatz 113383` ne produca 248; il numero di righe di `list` sia vicino a quello di `ps -el | wc -l`; PID 1 (`init/systemd`) appaia *secondo* nell'output di `tree` (subito dopo il task swapper PID 0).

Durante il test finale su VM Ubuntu 22.04 (kernel 5.15, architettura aarch64 via Multipass su macOS), ho riscontrato un bug nel parsing: tutti i comandi restituivano «Error: empty command». L'analisi ha rivelato un errore nel `dispatch_command` (uso scorretto di `strsep`, descritto nella Parte III). Dopo la correzione, tutti i comandi hanno prodotto output conforme all'atteso. Estratto reale dal `dmesg` della sessione di test:

```
kexplorer: module loaded (buffer_size=4096)
kexplorer: Collatz(27) - 112 nodes freed
kexplorer: Collatz(1) - 1 nodes freed
kexplorer: Collatz(6) - 9 nodes freed
kexplorer: module unloaded
kexplorer: module loaded (buffer_size=1024)
kexplorer: Collatz(999999999) - 362 nodes freed
```

Parte III — Diario dell'interazione con l'IA

13. Strumenti e modalità di utilizzo

Ho usato **Claude Opus 4.7** di Anthropic (interfaccia web `claude.ai`) come unico assistente di IA durante lo sviluppo. Le interazioni si sono concentrate su tre fronti:

- **Chiarimenti teorici:** per capire alcuni dettagli specifici (es. la differenza fra `task->state` e `task->__state`, il significato preciso di `container_of`, perché serve `INIT_LIST_HEAD` se uso poi `list_add_tail`).
- **Scrittura del codice:** ho dato in input i TODO della specifica e ho chiesto un'implementazione coerente con i commenti del professore. In tutti i casi ho riletto, modificato e in alcuni casi riscritto l'output prima di integrarlo.
- **Debugging:** per individuare quale flag GFP usare in `kmalloc`, per controllare l'uso corretto delle macro di iterazione delle linked list, e per ragionare sulla strategia del troncamento del buffer.

Non ho usato Copilot dentro l'editor perché mi sembrava troppo intrusivo per un progetto in cui volevo capire ogni riga.

14. Schede di interazione per fase

Scheda — Fase 1 (info, gcd, infrastruttura)

Strumento IA	Claude Opus 4.7
Obiettivo	Capire come creare un'entry <code>/proc</code> che supporti sia <code>read</code> che <code>write</code> , e implementare i comandi <code>info</code> e <code>gcd</code> .
Prompt principale	«Spiegami la differenza fra <code>file_operations</code> e <code>proc_ops</code> nelle versioni recenti del kernel Linux, e mostrami un esempio minimale di entry <code>/proc</code> con callback di lettura e scrittura.»
Risposta dell'IA	Risposta chiara con esempio compilabile. Ha spiegato che a partire dal kernel 5.6 le funzioni <code>proc</code> usano <code>struct proc_ops</code> al posto di <code>file_operations</code> .
Funzionava?	Sì, dopo aver allineato i nomi dei campi (<code>.proc_read</code> invece di <code>.read</code>).
Modifiche apportate	Adattato all'infrastruttura già fornita dal professore: il template <code>kexplorer.c</code> usava già <code>proc_ops</code> , quindi ho solo seguito il pattern.
Iterazioni successive	Una sola, per chiedere se <code>O666</code> fosse un permesso adatto e perché in molti tutorial si vede <code>O644</code> .
Cosa ho imparato	Che <code>proc_create()</code> richiede un'attenzione specifica alle permissions (<code>write</code> deve essere abilitato dal modo se voglio supportare la scrittura). Ho lasciato <code>O666</code> come nel template ma capendo cosa significa.

Scheda — Fase 2 (task)

Strumento IA	Claude Opus 4.7
Obiettivo	Implementare la conversione <code>PID → task_struct</code> e capire come gestire i PID namespace.
Prompt principale	«Qual è la differenza fra <code>find_pid_ns</code> , <code>find_vpid</code> e <code>find_task_by_vpid</code> ? E perché mi conviene la seconda?»
Risposta dell'IA	Spiegazione dettagliata dei PID namespace e del fatto che <code>find_vpid</code> opera nel namespace corrente del chiamante (più sicura quando il modulo viene caricato in un container).
Funzionava?	Sì.
Modifiche apportate	Ho aggiunto un controllo esplicito per PID negativo prima di passare il valore a <code>find_vpid</code> : la funzione del kernel ritorna NULL anche per input fuori range ma voglio un messaggio d'errore distinto per «PID negativo» e «PID non trovato».
Iterazioni successive	Due. La seconda per chiedere il discorso del campo <code>state</code> rispetto a <code>state</code> nei kernel recenti.
Cosa ho imparato	Il concetto di PID namespace, importante in contesti container, e il problema dei nomi rinominati nell'evoluzione del kernel (motivo per cui ho introdotto la macro <code>KE_TASK_STATE</code>).

Scheda — Fase 3 (list, tree)

Strumento IA	Claude Opus 4.7
Obiettivo	Implementare il DFS sull'albero dei processi e gestire il troncamento del buffer in modo elegante.
Prompt principale	«Come gestisco il troncamento del buffer durante una ricorsione DFS, evitando di scrivere il messaggio di troncamento più volte e senza interrompere la deallocazione delle risorse?»
Risposta dell'IA	Ha suggerito il pattern con flag statico a livello di file, <code>dfs_truncated</code> , controllato all'ingresso di ogni livello di ricorsione e nel loop sui figli. Pattern molto pulito.
Funzionava?	Sì.
Modifiche apportate	Ho aggiunto il <i>reserve</i> : prima di scrivere ogni nodo, mi assicuro che resti spazio sufficiente per il messaggio di troncamento. Senza questo, nel peggior caso il messaggio finale non ci starebbe.
Iterazioni successive	Una. Ho chiesto se <code>list_for_each</code> fosse safe in caso di modifiche concorrenti dell'albero (es. fork in corso da parte di altri task). La risposta è stata: no, ma il kernel usa RCU per leggerla in modo lock-free; per un modulo didattico interattivo il rischio è accettabile.
Cosa ho imparato	Il pattern del «riserva spazio» per gestire output limitati è generale, l'ho riusato anche in <code>cmd_list</code> e in <code>cmd_collatz</code> .

Scheda — Fase 4 (collatz)

Strumento IA	Claude Opus 4.7
Obiettivo	Costruire la sequenza di Collatz in una linked list del kernel e deallocarla correttamente.
Prompt principale	«Voglio iterare su una linked list del kernel e cancellare ogni nodo durante l'iterazione: c'è una macro specifica?»
Risposta dell'IA	Risposta puntuale: <code>list_for_each_entry_safe</code> , che mantiene un puntatore al nodo successivo prima di processare quello corrente, rendendo sicura la rimozione.

Funzionava?	Sì.
Modifiche apportate	Ho aggiunto un percorso di rollback nel caso <code>kmalloc</code> fallisca a metà della generazione: rilascio tutti i nodi già inseriti prima di ritornare con errore. Questo non era presente nella prima versione che mi era stata suggerita.
Iterazioni successive	Due. La seconda per capire se <code>INIT_LIST_HEAD(&node->list)</code> sia necessaria prima di <code>list_add_tail</code> : la risposta è no in stretto rigore (l'add riscrive comunque i puntatori), ma è una <i>buona pratica</i> per rendere lo stato esplicito e per prevenire bug se qualcuno cancella il nodo prima di inserirlo.
Cosa ho imparato	La differenza fra le varianti <i>safe</i> delle macro di iterazione, e il principio di «rollback totale» sui rami di errore.

15. Analisi critica e bilancio finale

Analisi critica del codice generato dall'IA

Il codice generato dall'IA è stato in generale *corretto sul piano sintattico e usava le API del kernel nei modi giusti*, ma presentava ricorrenti debolezze su cui sono dovuto intervenire:

- **Gestione del troncamento:** le prime proposte si limitavano a un controllo `if (output_len + written >= buffer_size)` dopo lo `snprintf`, che però lascia il buffer in stato «a metà» (`snprintf` ha già copiato fino al limite). Ho dovuto introdurre il pattern del *reserve* e lo scarto esplicito della riga troncata.
- **Percorsi di errore:** l'IA tendeva a non rilasciare le risorse parziali sui rami di errore (esempio classico: `kmalloc` fallisce a metà della sequenza di Collatz, ma i nodi già inseriti nella lista restavano allocati).
- **Distinzione fra errori di sistema e errori dell'utente:** spesso il codice usava `printk(KERN_ERR)` anche per «PID inesistente», che invece è un errore atteso dell'utente e va segnalato nel buffer di output, non nel log del kernel (che si riempirebbe di rumore).
- **Compatibilità di versione:** alcune proposte usavano `task->state` assumendo un kernel < 5.14; altre il contrario. La macro `KE_TASK_STATE` è stata una mia aggiunta dopo aver verificato il problema.
- **Bug nel parsing dei comandi (strsep):** il bug più significativo emerso durante il testing è stato nel `dispatch_command`. Il codice generato dall'IA usava `strsep(&rest, "\n")` per rimuovere il newline, e poi `strsep(&rest, " ")` per estrarre il comando. Ma dopo il primo `strsep`, `rest` punta alla parte dopo il newline (stringa vuota), non al testo utile che finisce in `token`. Risultato: ogni comando restituiva «Error: empty command». La correzione è una sola riga: `rest = token;` dopo la rimozione del newline. Un errore non banale: il codice era sintatticamente perfetto, compilava senza warning, e il bug si manifestava solo a runtime. È il tipo di errore che l'IA introduce perché non può eseguire il codice.

Bilancio finale

L'IA ha accelerato sicuramente il lavoro, soprattutto nella ricerca delle API kernel da usare e nel ricordare il nome esatto delle macro (`list_for_each_entry_safe` e `find_vpid` non sono nomi che ricorderei a mente). Allo stesso tempo, mi ha costretto a un atteggiamento critico: ogni suggerimento andava verificato sulla documentazione del kernel e contestualizzato nello specifico progetto. Senza questa rilettura, il codice avrebbe avuto problemi di memory management sui rami di errore e una gestione fragile del troncamento.

Se dovessi rifare il progetto userei l'IA fin da subito ma con una regola: *per ogni funzione che mi propone, prima di integrarla la riscrivo nello stile del progetto e ne traccio manualmente l'esecuzione su due o tre input*. È un'abitudine che protegge da bug subdoli (off-by-one nelle iterazioni, puntatori non inizializzati) che l'IA può introdurre senza che siano evidenti alla lettura.

Le fasi in cui avrei potuto fare a meno dell'IA sono essenzialmente la Fase 1 (infrastruttura

standard) e la validazione degli input — codice molto convenzionale che si scrive più velocemente a memoria. Per la Fase 3 (DFS e troncamento) e la Fase 4 (linked list intrusive) l'IA è stata utile per decidere il pattern; per le altre, la documentazione del kernel sarebbe stata sufficiente.

Conclusioni

Il progetto ha raggiunto tutti gli obiettivi didattici dichiarati nella specifica: scrittura di un modulo kernel funzionante, uso dell'infrastruttura `/proc` per la comunicazione bidirezionale, attraversamento dell'albero dei processi sia lineare sia in DFS, uso delle strutture dati del kernel (linked list intrusive) e gestione esplicita della memoria con verifica dell'assenza di leak.

Oltre a ciò, mi sono trovato a riflettere su aspetti che la specifica non sollevava direttamente ma che sono emersi durante lo sviluppo: la **portabilità fra versioni di kernel** (`state` → `__state`), la **resilienza ai casi limite** (buffer troppo piccolo, allocazione fallita a metà), la **distinzione fra log del kernel e output utente**. Sono tutti aspetti che differenziano il codice didattico da quello «da produzione», e su cui mi auguro di poter continuare a lavorare nei progetti successivi del corso.

Riferimenti

- Silberschatz, Galvin, Gagne. «Operating System Concepts», 10ª edizione, Wiley, 2018 — capitoli 2 e 3 (Programming Projects).
- Materiali del corso SOSI: *SOSILAB_KernelExplorer.pdf*, *SOSILAB_modulilinux.pdf*, *SOSILAB_PROCFS.pdf*, *SOSILAB_make.pdf*, *SOSILAB_lineeguidarelazioneprogetti.pdf*.
- Linux Kernel Module Programming Guide: <https://tldp.org/LDP/lkmpg/2.6/html/> (con riserva: alcuni esempi datati, da adattare ai kernel moderni).
- Documentazione del filesystem `proc`: <https://www.kernel.org/doc/html/latest/filesystems/proc.html>
- Linux kernel API — linked lists: <https://www.kernel.org/doc/html/latest/core-api/kernel-api.html>
- Robert Love, «Linux Kernel Development», 3ª edizione, Addison-Wesley, 2010 — riferimento storico per le strutture dati del kernel; alcune API sono cambiate ma i concetti restano validi.